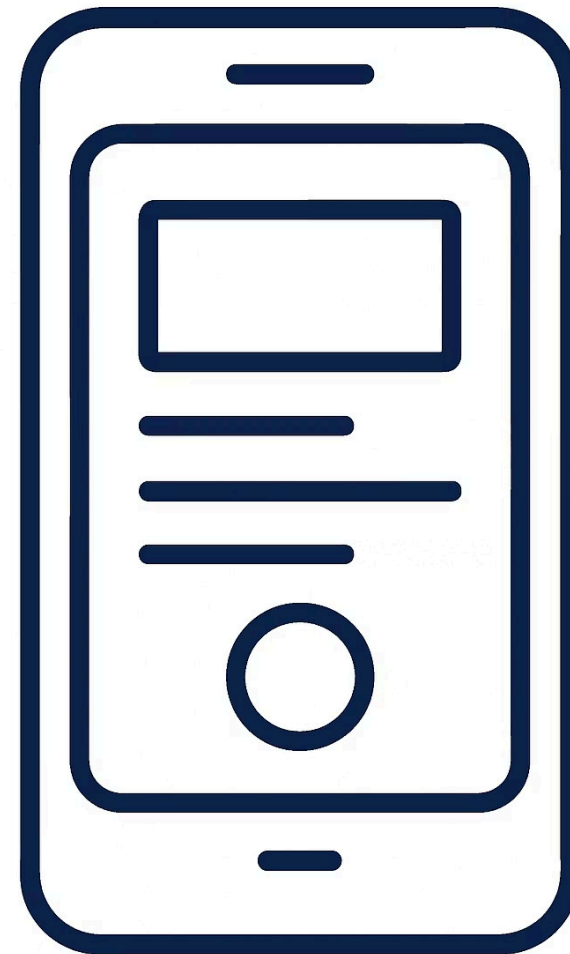


MÓDULO 1 · T1

Fundamentos: Entorno y Expo

🚀 *De cero a tu primera app nativa en minutos*

Carlos Rodríguez Contreras · GitHub: [karrocon](#)



Objetivos del módulo

Al terminar esta sesión serás capaz de configurar tu entorno, crear un proyecto real con Expo y ver tu primera app corriendo en un dispositivo o emulador.

1

Instalar el entorno

Node.js, Expo CLI y un emulador o dispositivo físico listos para trabajar.

2

Crear el proyecto

Scaffolding instantáneo con `create-expo-app` y plantilla TypeScript.

3

Entender la estructura

Conocer cada carpeta y archivo generado para moverte con soltura desde el primer día.

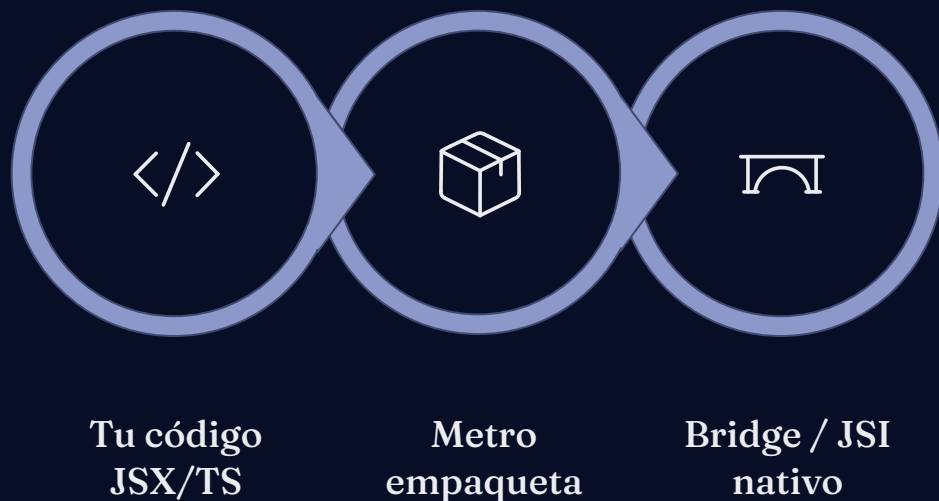
4

Ejecutar la app

Lanzarla en el emulador y previsualizarla con Expo Go desde tu móvil.

¿Qué es React Native?

React Native no es un WebView disfrazado de app: compila directamente a **componentes nativos reales** de iOS y Android. Usas el mismo modelo de componentes que ya conoces de React, pero el resultado es indistinguible de una app escrita en Swift o Kotlin.



Un único código base genera binarios nativos para ambas plataformas, lo que reduce drásticamente el tiempo de desarrollo y mantenimiento.



¿Qué es Expo?

Expo es la plataforma que elimina la fricción inicial de React Native. Puedes tener tu primera app corriendo en minutos, sin tocar Xcode ni Android Studio.



create-expo-app

Scaffolding instantáneo con una sola línea de comando. Elige entre plantillas JavaScript o TypeScript.



Expo Go

Previsualiza tu app en tiempo real escaneando un QR. Sin compilar, sin esperas.



EAS (Expo Application Services)

Build y deploy en la nube. Genera ficheros `.ipa` y `.apk` sin hardware Apple ni Android.

✓ No necesitas Xcode ni Android Studio para empezar. Expo Go lo gestiona todo en tu dispositivo.

Instalación paso a paso

Tres comandos son suficientes para tener tu proyecto listo. Asegúrate de tener **Node.js** $\geq$ **18** instalado antes de empezar (node --version).

1. Verificar Node.js

```
node --version # debe mostrar v18 o superior
```

2. Crear el proyecto con plantilla TypeScript

```
npx create-expo-app@latest MiPokedex --template blank-typescript
```

3. Entrar al directorio e iniciar el servidor

```
cd MiPokedex
```

```
npx expo start
```

Expo Go (recomendado)

Instala Expo Go en tu móvil iOS o Android, escanea el QR que aparece en terminal y tu app se abre al instante.

Emulador

Pulsa **a** en la terminal para abrir Android Studio Emulator, o **i** para el simulador de iOS (solo macOS).

Estructura de carpetas

Al generar el proyecto con `create-expo-app` obtienes una estructura limpia y predecible. Al principio solo tocarás dos elementos: **App.tsx** y la carpeta **assets/**.

MiPokedex/

- |—— app.json ← nombre, versión, iconos y splash
- |—— App.tsx ← punto de entrada de la aplicación
- |—— assets/ ← imágenes, fuentes y pantalla de carga
 - | |—— icon.png
 - | |—— splash.png
- |—— babel.config.js ← transpilación con Babel
- |—— package.json ← dependencias del proyecto
- |—— tsconfig.json ← configuración de TypeScript

→ app.json

Centraliza toda la configuración de la app: nombre, versión, orientación, permisos e iconos.

→ App.tsx

El componente raíz. React Native renderiza este archivo al arrancar.

→ assets/

Guarda aquí imágenes locales, fuentes personalizadas y el splash screen.

Tu primer componente

Este es el código mínimo de un componente React Native. Observa que usamos `View`, `Text` y `StyleSheet` en lugar de `div`, `p` y `CSS`.

```
// App.tsx
import { View, Text, StyleSheet } from 'react-native';

export default function App() {
  return (
    <View style={styles.container}>
      <Text style={styles.title}>¡Hola Pokédex!</Text>
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    alignItems: 'center',
    justifyContent: 'center',
  },
  title: {
    fontSize: 24,
    fontWeight: '700',
  },
});
```

View

Equivalente al `div` de la web. Contenedor genérico con Flexbox.

Text

Todo el texto debe ir dentro de `<Text>`. No hay texto suelto en JSX nativo.

StyleSheet.create

Optimiza los estilos en el hilo nativo. Mejor rendimiento que objetos inline.

Añadir una imagen

El componente `Image` de React Native soporta dos fuentes: URLs remotas y archivos locales. Siempre es obligatorio especificar **width** y **height** para que la imagen se renderice correctamente.

Imagen remota (URL)

```
import { Image } from 'react-native';

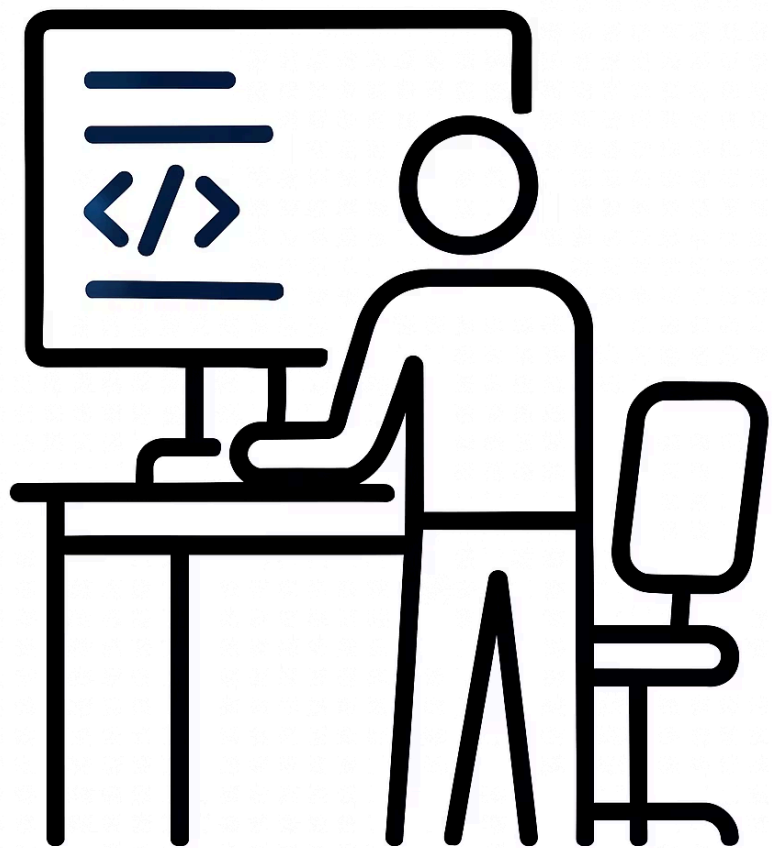
<Image
  source={{
    uri: 'https://raw.githubusercontent.com/
PokeAPI/sprites/master/
sprites/pokemon/25.png'
  }}
  style={{ width: 96, height: 96 }}
/>
```

Imagen local (asset)

```
import { Image } from 'react-native';

<Image
  source={require('./assets/icon.png')}
  style={{ width: 96, height: 96 }}
/>
```

⚠ Omitir **width** y **height** hará que la imagen no aparezca. En React Native no existe inferencia automática de tamaño para imágenes remotas.



Tour por el starter — App.tsx

El archivo `starter/App.tsx` es tu punto de partida para el ejercicio. Los comentarios `// TODO` marcan exactamente los puntos donde debes implementar el código.

```
// starter/App.tsx
import { View, Text, StyleSheet } from 'react-native';

// TODO 1: añade un <Text> con el título de la app
// TODO 2: añade un <Image> con la imagen de un Pokémon
export default function App() {
  return (
    <View style={styles.container}>
      <Text>¡Hola Expo!</Text>
    </View>
  );
}
```

⚠ Trabaja siempre desde la carpeta `starter/`. La solución completa está en `solucion/` para que puedas compararla al terminar.

Solución — solucion/App.tsx

Así queda el componente una vez implementados los dos `TODO`. Fíjate en cómo se combinan `Image`, `Text` y `StyleSheet` para construir una pantalla presentable.

```
// solucion/App.tsx
import { View, Text, Image, StyleSheet } from 'react-native';

export default function App() {
  return (
    <View style={styles.container}>
      <Image
        source={{
          uri: 'https://raw.githubusercontent.com/PokeAPI/sprites/master/sprites/pokemon/other/official-artwork/25.png',
        }}
        style={styles.image}
      />
      <Text style={styles.title}>Mi Pokédex</Text>
      <Text style={styles.subtitle}>
        Explora los 151 originales
      </Text>
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    alignItems: 'center',
    justifyContent: 'center',
    backgroundColor: '#fff',
  },
  image: { width: 160, height: 160, marginBottom: 16 },
  title: { fontSize: 28, fontWeight: '700' },
  subtitle: { fontSize: 16, color: '#666', marginTop: 4 },
});
```

Ejercicio T01

Pon en práctica todo lo que has aprendido. Sigue estos pasos en orden y no olvides verificar el resultado en el emulador o en Expo Go antes de dar por terminado el ejercicio.

01

Crea el proyecto

Ejecuta `npx create-expo-app@latest MiPokedex --template blank-typescript` en tu terminal.

02

Abre el starter

Copia el contenido de `starter/App.tsx` y úsalo como base para tu implementación.

03

Personaliza el contenido

Añade un `<Text>` con tu nombre y un componente `<Image>` con el sprite de tu Pokémon favorito.

04

Aplica estilos

Usa `StyleSheet.create` para centrar el contenido, ajustar tamaños de fuente y dar margen a la imagen.

05

Verifica en dispositivo

Lanza con `npx expo start` y comprueba que todo se ve correctamente en el emulador o en Expo Go.

Resumen del módulo

Estos son los cinco conceptos clave que debes haber interiorizado al terminar la sesión T01. Llévalos contigo al siguiente módulo.

Concepto	Clave a recordar
React Native	Compila a componentes nativos reales, no a un WebView. Mismo modelo de componentes que React.
Expo	Simplifica el setup, el preview (Expo Go) y el build en la nube (EAS). Sin Xcode ni Android Studio.
View	El contenedor genérico de React Native. Equivalente al <code>div</code> de la web, con Flexbox incluido.
Text	Único componente válido para mostrar texto. No existe texto suelto fuera de <code><Text></code> .
Image	Acepta <code>uri</code> (URL remota) o <code>require()</code> (local). Siempre requiere <code>width</code> y <code>height</code> .
StyleSheet.create	Optimiza los estilos en el hilo nativo. Preferible a objetos de estilo inline para mejor rendimiento.

✅ ¡Enhorabuena! Ya tienes tu entorno configurado y tu primera app funcionando. En el próximo módulo exploraremos componentes, props y estado con React Native.